

Assignment 3

Student Name: Priyanshu

Enrollment/Roll Number: IoT-029

Course Name: Internet of Things (IoT)

Assignment Title:

Submission Date:

Ans35.

WeatherFetcher.ino

```
WeatherFetcher.ino    config.h
1  #include <WiFi.h>
2  #include <HttpClient.h>
3  #include <ArduinoJson.h>
4  #include <DHT.h>
5  #include "config.h"
6
7  // ----- DHT11 -----
8  #define DHTPIN 15
9  #define DHTTYPE DHT11
10
11 DHT dht(DHTPIN, DHTTYPE);
12
13 // Function declaration
14 void fetchWeather();
15
16 void setup() {
17     Serial.begin(115200);
18
19     dht.begin();
20
21     delay(3000);
22
23     // Connect to WiFi
24     WiFi.begin(WIFI_SSID, WIFI_PASSWORD);
25
26     Serial.print("Connecting to WiFi");
27 }
```

WeatherFetcher.ino

config.h

```
28     while (WiFi.status() != WL_CONNECTED) {
29         delay(500);
30         Serial.print(".");
31     }
32
33     Serial.println("\nWiFi Connected!");
34     Serial.print("IP Address: ");
35     Serial.println(WiFi.localIP());
36 }
37
38 void loop() {
39
40     fetchWeather();
41
42     Serial.println("\nWaiting 30 seconds...\n");
43
44     delay(30000);
45 }
46
47 void fetchWeather() {
48
49     if (WiFi.status() != WL_CONNECTED) {
50         Serial.println("WiFi Disconnected!");
51         return;
52     }
```

WeatherFetcher.ino

config.h

```
54 String url = "http://api.openweathermap.org/data/2.5/weather?q=" +
55 | | | | | String(CITY) + "," +
56 | | | | | String(COUNTRY_CODE) +
57 | | | | | "&appid=" + String(API_KEY) +
58 | | | | | "&units=metric";
59
60 HTTPClient http;
61
62 http.begin(url);
63
64 int httpCode = http.GET();
65
66 if (httpCode == HTTP_CODE_OK) {
67
68     String payload = http.getString();
69
70     DynamicJsonDocument doc(2048);
71
72     DeserializationError error = deserializeJson(doc, payload);
73
74     if (error) {
75         Serial.println("JSON Parsing Failed!");
76         http.end();
77         return;
78     }
```

WeatherFetcher.ino config.h

```
81     String city = doc["name"];
82     float apiTemp = doc["main"]["temp"];
83     int apiHumidity = doc["main"]["humidity"];
84     String description = doc["weather"][0]["description"];
85
86     // Local DHT11 Values
87     float localTemp = dht.readTemperature();
88     float localHumidity = dht.readHumidity();
89
90     if (isnan(localTemp) || isnan(localHumidity)) {
91         Serial.println("Failed to read DHT11!");
92         http.end();
93         return;
94     }
95
96     float tempDifference = localTemp - apiTemp;
97
98     Serial.println("-----");
99     Serial.println("          WEATHER COMPARISON");
100    Serial.println("-----");
101
102    Serial.print("City                : ");
103    Serial.println(city);
104
105    Serial.print("Weather                : ");
106    Serial.println(description);
107
108    Serial.print("API Temperature        : ");
109    Serial.print(apiTemp);
```

```

110     Serial.println(" °C");
111
112     Serial.print("DHT11 Temperature   : ");
113     Serial.print(localTemp);
114     Serial.println(" °C");
115
116     Serial.print("Temperature Difference : ");
117     Serial.print(tempDifference);
118     Serial.println(" °C");
119
120     Serial.print("API Humidity         : ");
121     Serial.print(apiHumidity);
122     Serial.println(" %");
123
124     Serial.print("DHT11 Humidity         : ");
125     Serial.print(localHumidity);
126     Serial.println(" %");
127
128     Serial.println("-----");
129 }
130
131 else {
132
133     Serial.print("HTTP Error Code: ");
134     Serial.println(httpCode);
135
136     String response = http.getString();
137     Serial.println(response);
138 }
139
140 http.end();
141 }

```

Config.h

WeatherFetcher.ino	config.h
1	<code>#ifndef CONFIG_H</code>
2	<code>#define CONFIG_H</code>
3	
4	<code>const char* WIFI_SSID = "Priyanshu's Galaxy A35";</code>
5	<code>const char* WIFI_PASSWORD = "12345678";</code>
6	
7	<code>const char* API_KEY = "c2cc5dc1acc3d0774e7d3fbd5a4dccd0";</code>
8	<code>const char* CITY = "Jammu";</code>
9	<code>const char* COUNTRY_CODE = "IN";</code>
10	
11	<code>#endif</code>

Output

```
Output  Serial Monitor  X
|Message (Enter to send message to 'ESP32 Dev Module' on 'COM19')
....
WiFi Connected!
IP Address: 10.35.94.4
-----
WEATHER COMPARISON
-----
City           : Jammu
Weather        : clear sky
API Temperature : 36.26 °C
DHT11 Temperature : 25.10 °C
Temperature Difference : -11.16 °C
API Humidity    : 38 %
DHT11 Humidity  : 69.80 %
-----

Waiting 30 seconds...
```

Ans36.

Feature	MQTT	HTTP
Architecture Pattern	Publish/Subscribe using a broker	Client/Server (Request-Response)
Data Transfer Model	Sends data only when required through topics	Client sends a request and waits for a response
Power Consumption	Low power consumption	Higher power consumption
Latency	Very low latency	Higher latency compared to MQTT
Use Case Suitability for IoT	Excellent for IoT devices, sensors, and real-time monitoring	Better for web applications, APIs, and occasional data exchange
Security Considerations	Supports TLS/SSL encryption, authentication, and access control through the broker	Supports HTTPS (SSL/TLS), authentication, and secure APIs

For a **1000-node smart agriculture system**, I would choose **MQTT**.

Reason:

- It uses very little bandwidth, making it suitable for large numbers of sensor nodes.
- It consumes less power, which helps battery-operated devices last longer.
- It provides low latency, allowing quick delivery of sensor readings and alerts.
- The publish/subscribe model makes communication efficient because devices only receive the data they subscribe to.
- A single MQTT broker can efficiently manage communication among thousands of devices, making the system scalable.

Ans37.

Wi-Fi security is very important in IoT because most smart devices send and receive data over the internet. If the network is not secure, attackers can steal information or even take control of the devices. One common attack is based on WPA2 vulnerabilities. Although WPA2 is a secure Wi-Fi standard, some weaknesses, such as the KRACK attack, have shown that attackers can exploit devices that have not been updated with security patches.

Another common attack is the Evil Twin attack. In this attack, a hacker creates a fake Wi-Fi network with the same name as a real one. If an IoT device connects to this fake network, the attacker may be able to capture data or steal login credentials.

To make ESP32 devices more secure, developers should use WPA3 whenever possible because it provides better protection than WPA2. It is also a good idea to use TLS encryption with certificate pinning, so the ESP32 communicates only with trusted servers. Keeping the firmware updated, using strong passwords, and disabling unnecessary services also improve security.

Hardcoding Wi-Fi passwords or API keys directly into the source code is risky. If someone gets access to the code or uploads it to a public GitHub repository, anyone can see those credentials and misuse them. A better approach is to store passwords in a separate configuration file or use secure storage, and never upload sensitive information to public repositories.

Ans38.

LPWAN stands for Low Power Wide Area Network. It is a type of wireless network designed for IoT devices that need to send small amounts of data over long distances while using very little battery power. LPWAN is commonly used in smart agriculture, smart cities, environmental monitoring, and asset tracking because devices can operate for several years on a single battery.

LoRa is one of the most popular LPWAN technologies. It achieves long-range communication by using a technique called the Spreading Factor (SF). A higher spreading factor allows the signal to travel farther and improves communication in areas with weak signal strength. Although increasing the spreading factor reduces the data transmission speed, it helps maintain reliable communication over long distances while keeping power consumption low.

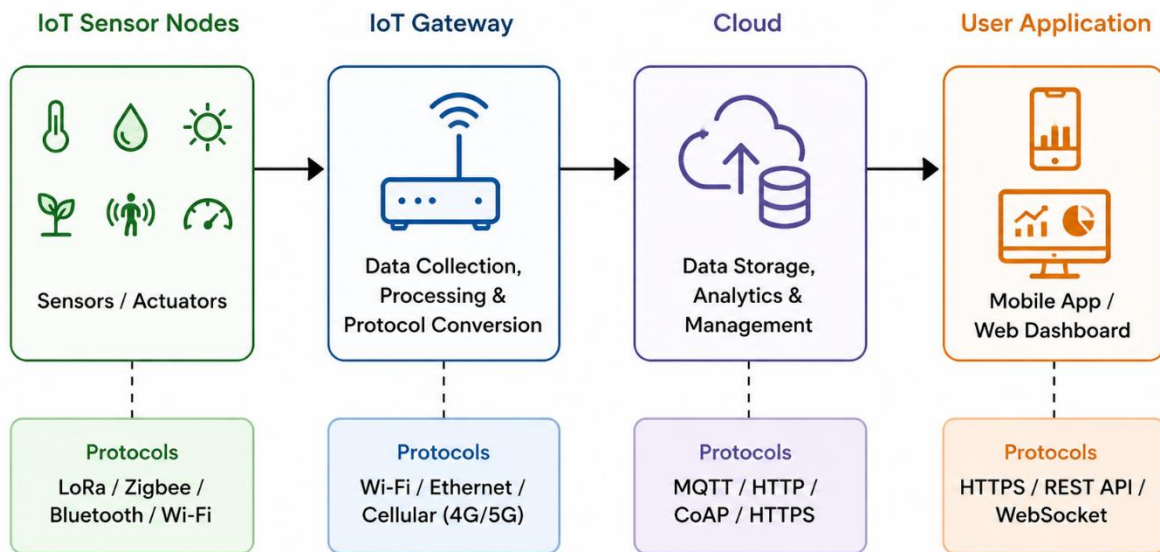
When compared with other wireless technologies, LoRa provides the longest communication range, often several kilometers, and is suitable for outdoor IoT applications. Zigbee has a shorter range but supports mesh networking, making it useful for home automation and industrial control. Z-Wave is mainly used in smart home devices and also has a shorter range than LoRa.

A real-world example in India is the use of LoRa-based smart agriculture solutions in states such as Maharashtra and Karnataka. Farmers use LoRa sensors to monitor soil moisture, temperature, and water levels in their fields. The collected data helps them make better irrigation decisions, reduce water wastage, and improve crop productivity.

Ans39.

An **IoT gateway** is a device that acts as a bridge between IoT sensor nodes and the cloud. It collects data from sensors, processes or filters it if needed, and then sends it to the cloud. It can also receive commands from the cloud and forward them to the connected IoT devices. An IoT gateway improves security, reduces network traffic, and allows devices using different communication protocols to work together.

Diagram



Protocols Used at Each Layer

Layer	Common Protocols
IoT Sensor Nodes → Gateway	LoRa, Zigbee, Bluetooth, Wi-Fi
Gateway → Cloud	MQTT, HTTP/HTTPS, CoAP
Cloud → User Application	HTTPS, REST API, WebSocket

Ans40.

QoS (Quality of Service) in MQTT defines how reliably a message is delivered between the publisher and the subscriber. MQTT provides three QoS levels.

QoS 0 – At Most Once

The message is sent only once without any confirmation from the receiver. If the message is lost, it is not sent again. This is the fastest method and uses the least bandwidth, but delivery is not guaranteed.

QoS 1 – At Least Once

The sender waits for an acknowledgment from the receiver. If no acknowledgment is received, the message is sent again. This guarantees that the message reaches the receiver, but it may be delivered more than once.

QoS 2 – Exactly Once

This is the highest level of reliability. It uses a four-step handshake to ensure that the message is delivered only once. It is slower and consumes more bandwidth but prevents duplicate messages.

For an **IoT water level monitoring system**, I would choose **QoS 1**

Reason:

In a water level monitoring system, it is important that sensor readings reach the server reliably. If a reading is accidentally sent twice, it usually does not cause a major problem because the latest value can be processed. However, losing a water level reading could result in incorrect monitoring or delayed alerts. QoS 1 provides a good balance between reliability, speed, and network usage, making it the most suitable choice for this application.

Ans41.

Objective

The aim of this project is to design an IoT-based Smart Home Air Quality Monitoring System that continuously monitors indoor air quality using an MQ-2 gas sensor and measures temperature and humidity using a DHT11 sensor. The system alerts users through an RGB LED, buzzer, and Bluetooth notifications whenever the air quality becomes unsafe. It is also capable of working without an internet connection because Bluetooth communication is used for local alerts.

Components Used

- ESP32 Development Board
- MQ-2 Gas Sensor (*Simulated using a potentiometer in Wokwi*)
- DHT11 Sensor (*DHT22 used in Wokwi for simulation*)
- RGB LED
- Active Buzzer
- Bluetooth Serial
- Jumper Wires
- Breadboard

Working Principle

The ESP32 continuously reads the gas level from the MQ-2 sensor and the temperature and humidity from the DHT11 sensor. Based on the gas sensor reading, the system classifies the air quality into three levels: **Safe, Moderate, and Danger**.

If the air quality is safe, the green LED glows and no alarm is generated. When the smoke level increases, the LED changes to yellow and the buzzer gives a warning beep. If the smoke level becomes dangerous, the LED turns red and the buzzer produces a faster alarm. At the same time, warning messages are sent to a mobile phone through Bluetooth. All sensor readings are also logged on the Serial Monitor in CSV format for easy monitoring and analysis.

Threshold Design

MQ-2 Reading	Air Quality	LED	Buzzer
0 – 299	Safe	Green	OFF
300 – 599	Moderate	Yellow	Slow Beep
600 – 4095	Danger	Red	Fast Beep

Sensor Calibration

Since the MQ-2 sensor is not available in Wokwi, a potentiometer is used to simulate the gas sensor. By rotating the potentiometer, different smoke levels are generated. During testing, the analog values were divided into three ranges representing safe, moderate, and dangerous air quality. These threshold values can be adjusted according to the actual MQ-2 sensor readings in a real hardware implementation.

CSV Output Format

Timestamp, MQ-2 Value, Temperature, Humidity, Severity

```
00:01,220,23.4,49,SAFE
00:02,430,23.5,50,MODERATE
00:03,765,23.7,49,DANGER
```

Bluetooth Alert Messages

SAFE

SAFE
Gas:210
Temp:22.8°C
Humidity:51%

MODERATE

WARNING
Moderate Smoke Detected

Gas=430

Check Fireplace

DANGER

DANGER!!

Smoke Level Critical

Gas=765

Open Windows

Leave Room if Necessary

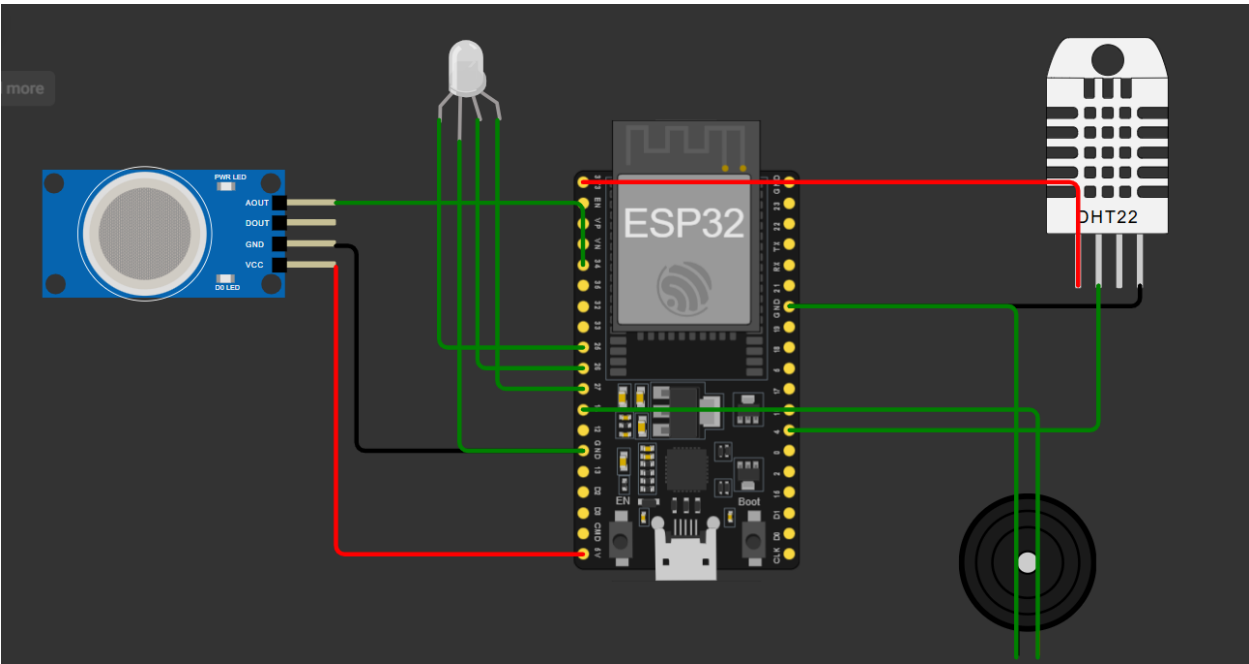
Advantages

- Monitors indoor air quality in real time.
- Provides visual, audio, and Bluetooth alerts.
- Works without an internet connection.
- Stores sensor data in CSV format for monitoring.
- Can be used in homes, offices, and small industries.

Applications

- Smart homes
- Indoor air quality monitoring
- Fire and smoke detection
- Warehouses
- Offices and commercial buildings

more



Ans42.

Objective

The objective of this project is to automate irrigation for a small saffron farm by monitoring soil moisture, temperature, and weather conditions. The system waters the plants only when the soil is dry and avoids watering during rainy or overcast conditions. It also allows the farmer to manually control watering through a push button while logging all watering events to Google Sheets.

Components Used

- ESP8266 (NodeMCU)
- Soil Moisture Sensor (*Simulated using Potentiometer in Wokwi*)
- LDR (Rain Proxy)
- Relay Module (*Simulated using LED*)
- Push Button
- 16×2 I2C LCD
- Wi-Fi Connection
- Google Sheets (via IFTTT/Google Apps Script)

Working Principle

The ESP8266 continuously checks the soil moisture level at regular intervals. Based on the sensor reading, the soil is classified as **Dry**, **Optimal**, or **Wet**.

Before turning on the water pump, the system checks two additional conditions:

- The **LDR sensor** detects whether the sky is overcast, which acts as a rain indicator.
- The **OpenWeatherMap API** checks if rain is expected or currently occurring.

The pump is activated **only when the soil is dry and there is no indication of rain**. A push button allows the farmer to manually start watering whenever required. After manual activation, the system enters a **5-minute lockout period** to prevent repeated watering.

The current soil moisture, operating mode, and last watering time are displayed on the **16×2 LCD**, while every watering event is logged to **Google Sheets** through Wi-Fi.

Threshold Design

Soil Moisture	Condition	Pump Status
0–29%	Dry	ON (<i>If no rain is detected</i>)
30–70%	Optimal	OFF
71–100%	Wet	OFF

Rain Detection Logic

The system uses two methods to avoid unnecessary watering:

- **LDR Sensor:** Detects low light or overcast conditions, which act as a rain proxy.
- **Weather API:** Fetches live weather information from OpenWeatherMap to check if it is raining or rain is expected.

The pump is turned **OFF** if either of these conditions indicates rain.

Manual Override

A push button is provided for manual watering.

- Pressing the button immediately starts the pump.
- After manual activation, the system ignores further button presses for **5 minutes**.
- This lockout prevents accidental repeated watering and protects the saffron crop from overwatering.

Google Sheets Logging

Every watering event is automatically logged to Google Sheets.

Time	Moisture	Rain	Pump	Mode
10:00	25%	No	ON	AUTO
10:30	55%	No	OFF	AUTO
11:00	24%	Yes	OFF	AUTO

	A	B	C	D	E	F
1	Time	Moisture (%)	Rain	Pump	Mode	Remarks
2	17/05/2025 10:00:09	25	No	ON	AUTO	Soil Dry
3	17/05/2025 10:30:12	55	No	OFF	AUTO	Optimal
4	17/05/2025 11:00:11	24	Yes	OFF	AUTO	Rain Detected
5	17/05/2025 11:30:10	22	No	ON	MANUAL	Manual Override
6						
7						

+ ≡ Sheet1 ▾

```

10:00:12 Soil Moisture : 24% (DRY)
10:00:12 LDR : Bright (No Overcast)
10:00:13 Weather API : No Rain
10:00:13 Pump : ON
10:00:13 Mode : AUTO
10:00:13 Data sent to Google Sheets
-----
10:30:12 Soil Moisture : 65% (OPTIMAL)
10:30:12 LDR : Overcast Detected
10:30:13 Weather API : Rain Detected
10:30:13 Pump : OFF
10:30:13 Mode : AUTO
10:30:13 Data sent to Google Sheets

```

Ans43.

Objective

The objective of this project is to develop a secure IoT-based smart door lock system for a university hostel. Instead of using physical keys, students enter their **4-digit ID** through a keypad. The system then generates a **6-digit One-Time Password (OTP)** and sends it to the student's phone via Bluetooth. The door unlocks only if the correct OTP is entered within **30 seconds**, providing an additional layer of security.

Components Used

- ESP32
- 4×4 Matrix Keypad
- 16×2 LCD (*or 0.96" OLED Display*)
- HC-05 Bluetooth Module (*or ESP32 Built-in Bluetooth*)
- Servo Motor (SG90)
- Jumper Wires
- Breadboard

Working Principle

When the system starts, the LCD prompts the user to enter a **4-digit student ID** using the keypad. As each digit is entered, it is displayed as an asterisk (*) to keep the ID hidden.

After validating the ID, the ESP32 generates a random **6-digit OTP** and immediately sends it to the student's phone through Bluetooth. At the same time, a **30-second countdown timer** starts using the `millis()` function, ensuring the program remains responsive without using `delay()`.

The student enters the received OTP using the keypad. If the entered OTP matches the generated OTP within the time limit, the servo motor rotates **90°**, simulating the unlocking of the hostel door. If the OTP is incorrect or the timer expires, access is denied, and the servo remains locked.

Every login attempt, whether successful or failed, is recorded in the Serial Monitor for future monitoring.

Program Logic

Step 1 – Student ID Verification

- Student enters a 4-digit ID.
- Digits are displayed as * * * * for privacy.

Step 2 – OTP Generation

- ESP32 generates a random 6-digit OTP.
- OTP is sent to the student's phone through Bluetooth.

Step 3 – Timer Starts

- A **30-second countdown** begins using millis().
- No delay() is used.

Step 4 – OTP Verification

- Student enters the OTP.
- If it matches within 30 seconds, the door unlocks.
- Otherwise, access is denied.

Step 5 – Door Control

- Servo rotates **90°** for successful authentication.
- Servo returns to **0°** after a few seconds.

Step 6 – Logging

Every attempt is logged to the Serial Monitor.

Simulation

▶

+

⋮

123AB

456BC

789CD

*0#D

ESP32

Enter ID:

===== SMART DOOR LOCK SYSTEM =====

System Started...

[12:00:01] Enter 4-digit Student ID:

ID Entered: 2401

Generating OTP...

OTP Generated: 583214

OTP sent via Bluetooth

Enter OTP (Valid for 30 seconds):

OTP Entered: 583214

Access Granted!

Door Unlocked (Servo 90°)

Door Locked (Servo 0°)

=====

⏮

⏪

⏴

⏵

⏭

⏹

⏶

Emergency calls only • Jio



10:47 Mon, 6 Jul

Live notifications

Prianshu Airwave Max 5 Media output

LONDON THUMAKDA
HA KAKKAR, AMIT TRIVEDI and ANVITA DUTT GUPTAN

01:28 03:50



Smart Door Lock 10:47 AM



Your OTP is: 583214

Valid for 30 seconds. Do not share.



Smart Door Lock 10:47 AM



Access Granted

Door unlocked successfully.



Smart Door Lock 10:46 AM



Incorrect OTP Entered

Access denied. Please try again.



Smart Door Lock 10:45 AM



OTP Expired

Time limit exceeded. Please try again.



Smart Door Lock 10:45 AM



Login Attempt Logged

ID: 2401 | Status: Failed

Notification settings

Ans44.

Objective

The objective of this project is to develop a low-cost IoT-based security system for a small shop. The system detects motion using a PIR sensor during configured shop closing hours. When motion is detected, it activates a buzzer with three alarm levels, flashes LEDs sequentially to simulate a security light, logs the event with timestamps on the Serial Monitor, and sends an alert to the shop owner through a Telegram bot using the ESP32 Wi-Fi module.

Components Used

- ESP32 Development Board
- PIR Motion Sensor
- Potentiometer (Sensitivity Adjustment)
- Active Buzzer
- 3 LEDs (LED Strip Simulation)
- Wi-Fi Connection
- Telegram Bot
- Jumper Wires
- Breadboard

Working Principle

The ESP32 continuously monitors the PIR sensor for motion. The shop owner can configure the monitoring period using the Serial Monitor by entering a command such as:

```
SET_HOURS 22 06
```

This configures the system to operate between **10:00 PM and 6:00 AM**.

A potentiometer is used to adjust the PIR sensor sensitivity by changing the motion detection threshold. If motion is detected during the configured security period, the system immediately activates the alert sequence.

The alert begins with a **Warning** stage, where the buzzer emits slow beeps and LEDs flash slowly. If motion continues, the system escalates to the **Alarm** stage with faster beeps and quicker LED flashing. If motion still persists, it enters the **Urgent** stage, producing a continuous buzzer sound and rapid LED flashing.

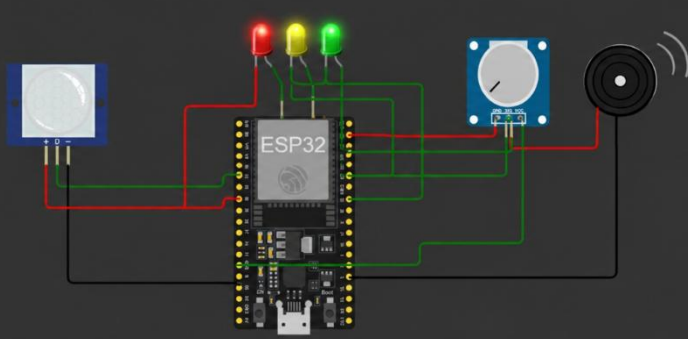
At the same time, the ESP32 sends a notification to the owner's Telegram account and records the event along with the timestamp in the Serial Monitor.

Pin Connections

Component	ESP32 Pin
PIR Sensor Output	GPIO 34
Potentiometer	GPIO 35
LED 1	GPIO 18
LED 2	GPIO 19
LED 3	GPIO 21
Active Buzzer	GPIO 22

Alert Escalation

Motion Duration	Level	LED Pattern	Buzzer
0–5 sec	Warning	Slow Sequential Flash	Slow Beep
5–10 sec	Alarm	Medium Flash	Fast Beep
More than 10 sec	Urgent	Rapid Flash	Continuous Tone



The schematic shows an ESP32 microcontroller board at the center. It is connected to a PIR motion sensor (blue square) on the left via two green wires. Three LEDs (red, yellow, green) are connected to the top of the board with green wires. A buzzer (blue square) and a speaker (black circle) are connected to the right side of the board with green wires. A black wire connects the bottom of the board to the speaker.

Security Window : 22:00 - 06:00

=====

System Armed

Monitoring Motion...

=====

Time : 22:14:32	Motion Detected	Sensitivity : 74%	Alarm Level : WARNING	Telegram Alert Sent
Time : 22:14:39	Motion Still Active	Alarm Level : ALARM	LED Pattern : FAST	
Time : 22:14:48	Motion Persistent	Alarm Level : URGENT	Continuous Alarm Activated	Telegram Message Sent

=====

y • Jio True5G | Jio



11:38 Tue, 7 Jul



Missed Notifications 11:34 am



Security Bot Bot

11:34 am



 Motion Detected
Shop: Main Entrance
Time: 22:14
Alarm Level: WARNING



Security Bot Bot

11:34 am



 Motion Still Active
Shop: Main Entrance
Alarm Level: ALARM
Please Check CCTV



Security Bot Bot

11:35 am



 URGENT ALERT 
Continuous Motion
Immediate Action Required
Shop: Main Entrance

Notification settings

Clear

Ans45.

Objective

Develop an automated greenhouse monitoring system for tulip and rose cultivation in Kashmir. The system continuously monitors environmental conditions and automatically controls the greenhouse to maintain ideal plant growth.

Ideal Growing Conditions

Parameter	Range
Temperature	18°C – 22°C
Humidity	60% – 70%
Light Duration	12 Hours/Day

Components Used

Component	Quantity
ESP32	1
DHT11 Sensor	1
LDR Module	1
10kΩ Resistor (if raw LDR)	1
16×2 LCD (I2C)	1
LEDs (Relay Simulation)	3
220Ω Resistors	3
Breadboard	1
Jumper Wires	As required

Functional Description

Temperature Monitoring

- DHT11 continuously measures temperature.
- If temperature drops below **18°C**, Heater turns ON.
- If temperature rises above **22°C**, Ventilation Fan turns ON.
- Between 19–21°C everything remains OFF.

Humidity Monitoring

- Maintain humidity between **60–70%**
- Humidity is displayed on LCD.
- No actuator required unless specified.

Light Monitoring

- LDR continuously measures light intensity.
- Running average of last **10 readings** is calculated.
- If average light is insufficient:
 - Grow Light turns ON.
- Otherwise:
 - Grow Light remains OFF.

Bang-Bang Control with Hysteresis

Instead of turning devices ON/OFF every second, hysteresis is used.

Example

Heater

- ON below 18°C
- OFF above 19°C

Fan

- ON above 22°C
- OFF below 21°C

This avoids continuous switching.

LDR Module

DHT22

ESP32

HEATER

FAN

GROW LIGHT

Temp: 20.4 C
Hum : 64 %

=== Smart Greenhouse Controller ===
System Started Successfully!
Monitoring Environment...

[11:32:10] Temp: 20.4 C | Hum: 64 % | Light: 712 (Avg: 705) | Heater: OFF | Fan: OFF | Grow Light: OFF | Status: OPTIMAL
[11:32:25] Temp: 17.6 C | Hum: 65 % | Light: 410 (Avg: 415) | Heater: ON | Fan: OFF | Grow Light: ON | Status: HEATING
[11:32:40] Temp: 22.8 C | Hum: 69 % | Light: 755 (Avg: 740) | Heater: OFF | Fan: ON | Grow Light: OFF | Status: COOLING
[11:32:55] Temp: 20.1 C | Hum: 63 % | Light: 610 (Avg: 600) | Heater: OFF | Fan: OFF | Grow Light: OFF | Status: OPTIMAL

Publishing data to MQTT Broker...

Ans46.

Objective

The objective of this project is to develop a low-cost IoT-based flood monitoring system for rivers such as the Tawi and Chenab. The system continuously measures the distance between an ultrasonic sensor and the water surface. As the water level rises, the measured distance decreases. Based on predefined thresholds, the system generates different warning levels, activates alarms, and broadcasts the alert wirelessly. A web dashboard displays the live water level graph for continuous monitoring.

Components Used

- ESP32 Development Board
- HC-SR04 Ultrasonic Sensor
- 3 LEDs (Green, Yellow, Red)
- Active Buzzer
- Wi-Fi / Bluetooth
- Web Dashboard (Chart.js)
- Breadboard
- Jumper Wires

Working Principle

The HC-SR04 ultrasonic sensor is mounted above the river (or glass tank) and continuously measures the distance to the water surface. When the water level rises, the measured distance decreases.

To reduce incorrect readings caused by ultrasonic noise, the ESP32 calculates a **rolling average of the last 10 sensor readings** before determining the water level.

The measured distance is classified into three alert levels:

- **Normal:** Water is more than **30 cm** below the sensor.
- **Warning:** Water is between **15 cm and 30 cm** below the sensor.

- **Critical:** Water is less than **15 cm** below the sensor.

Each alert level activates a different LED and buzzer pattern. The ESP32 also broadcasts the current alert level over Wi-Fi or Bluetooth as a simple text message. A web dashboard created using **Chart.js** displays a live graph of the river water level.

Pin Connections

Component	ESP32 Pin
HC-SR04 Trigger	GPIO 5
HC-SR04 Echo	GPIO 18
Green LED	GPIO 19
Yellow LED	GPIO 21
Red LED	GPIO 22
Active Buzzer	GPIO 23

Alert Thresholds

Distance from Sensor	Water Level	Alert
Greater than 30 cm	Safe	NORMAL
15–30 cm	Rising	WARNING
Less than 15 cm	High	CRITICAL

Rolling Average Calculation

To improve accuracy, the ESP32 stores the last **10 ultrasonic distance readings**.

Example:

32
31
30
31
30
29

31
30
30
29

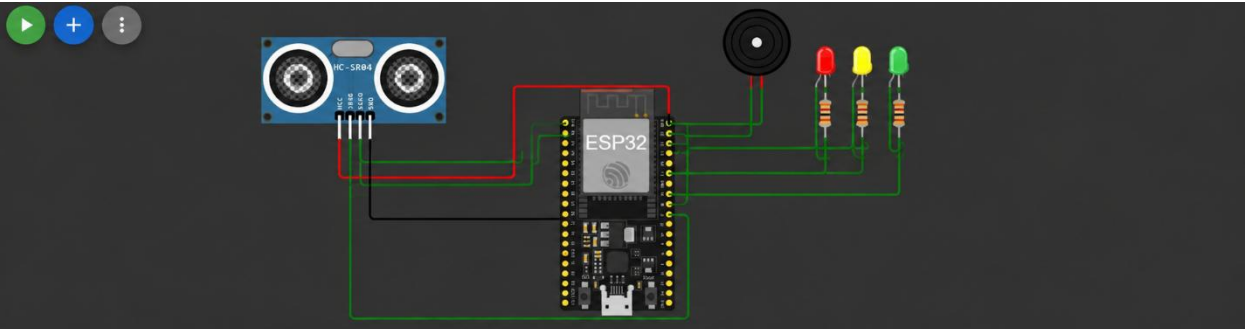
Average Distance

30.3 cm

This average is used instead of a single sensor reading, reducing false alarms caused by sensor noise.

LED & Buzzer Patterns

Alert	LED	Buzzer
NORMAL	Green	One short beep every 5 seconds
WARNING	Yellow	Two beeps every second
CRITICAL	Red	Continuous rapid beeping



```
=== River Water Level Alert System ===
ESP32 Initialized
Wi-Fi Connected Successfully
Monitoring Water Level... Press Ctrl+C to stop

[11:23:10] Distance: 36.8 cm | Avg(10): 36.5 cm | Level: NORMAL | Green LED: ON | Yellow LED: OFF | Red LED: OFF | Buzzer: OFF | Broadcast: NORMAL
[11:23:20] Distance: 24.3 cm | Avg(10): 24.7 cm | Level: WARNING | Green LED: OFF | Yellow LED: ON | Red LED: OFF | Buzzer: SLOW BEEP | Broadcast: WARNING
[11:23:30] Distance: 11.8 cm | Avg(10): 11.6 cm | Level: CRITICAL | Green LED: OFF | Yellow LED: OFF | Red LED: ON | Buzzer: CONTINUOUS | Broadcast: CRITICAL
[11:23:40] Distance: 28.6 cm | Avg(10): 29.1 cm | Level: WARNING | Green LED: OFF | Yellow LED: ON | Red LED: OFF | Buzzer: SLOW BEEP | Broadcast: WARNING
[11:23:50] Distance: 34.2 cm | Avg(10): 33.7 cm | Level: NORMAL | Green LED: ON | Yellow LED: OFF | Red LED: OFF | Buzzer: OFF | Broadcast: NORMAL

System Running...
```

Dashboard

Live Data

Alert Log

Settings

River Water Level Alert System

Connected (E)

Current Water Distance

22.6 cm

from sensor to water surface

Alert Level

WARNING

Water level is rising

Last Updated

11:24:35 AM

07 July 2025

System Status

ONLINE

All systems operational

Water Level (Distance) Over Time

Alert Level Guide

NORMAL

> 30 cm

Water level is safe

WARNING

15 - 30 cm

Water level is rising. Stay alert!

CRITICAL

< 15 cm

Danger! Take immediate action!

Recent Readings

Time	Distance (cm)	Average (10) (cm)	Level	Status
11:24:30 AM	22.6	22.8	WARNING	RISING
11:24:20 AM	23.4	23.1	WARNING	RISING
11:24:10 AM	24.1	23.6	WARNING	RISING
11:24:00 AM	25.0	24.0	WARNING	RISING
11:23:50 AM	26.2	24.6	WARNING	RISING

System Info

Device: ESP32

IP Address: 192.168.4.1

Uptime: 00:12:45

Wi-Fi: Connected

Ans47.

Objective

The objective of this project is to develop a simulated health monitoring system using a MAX30102 pulse oximeter sensor or an LDR as a proxy sensor. The system continuously monitors heart rate (HR) and blood oxygen saturation (SpO₂), applies data smoothing using a rolling average, displays readings every two seconds, and alerts the user when abnormal values are detected. This project is intended for educational purposes only and is not a medical device.

Components Used

Option 1 (Recommended)

- ESP32
- MAX30102 Pulse Oximeter Sensor
- 16×2 I2C LCD
- Red LED
- Jumper Wires

Option 2 (Simulation)

- ESP32
- LDR
- 10kΩ Resistor
- 16×2 I2C LCD
- Red LED
- Jumper Wires

Working Principle

The MAX30102 sensor measures changes in blood flow using infrared and red LEDs to estimate heart rate and oxygen saturation. If the MAX30102 is unavailable, an LDR can be used to simulate fluctuating sensor readings.

To reduce noise and sudden fluctuations, the ESP32 stores the last five readings and calculates a rolling average before displaying the values.

The readings are updated every two seconds on the LCD and Serial Monitor. If the heart rate falls below 50 bpm, rises above 120 bpm, or the oxygen saturation drops below 94%, the system activates a red LED warning indicator.

All readings are logged to a CSV file reference that can be included in the project's GitHub repository.

Pin Connections (MAX30102)

MAX30102	ESP32
VIN	3.3V
GND	GND
SDA	GPIO21
SCL	GPIO22

Pin Connections (LDR Simulation)

Component	ESP32
LDR Output	GPIO34
Red LED	GPIO25

Alert Conditions

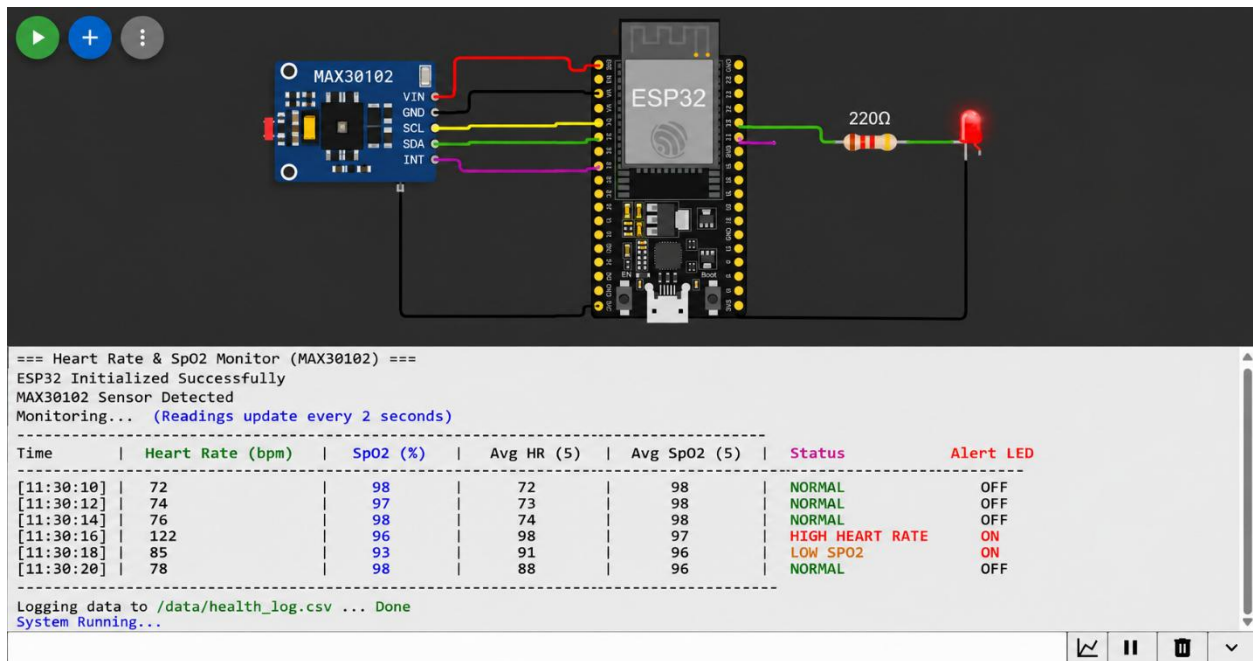
Parameter	Normal Range	Alert Condition
Heart Rate	50–120 bpm	<50 or >120 bpm
SpO ₂	94–100%	<94%

Advantages

- Continuous health monitoring simulation.
- Rolling average reduces sensor noise.
- Simple alert mechanism for abnormal values.
- Easy integration with IoT dashboards.
- Educational introduction to biomedical sensing.

Applications

- Health Monitoring Prototypes
- IoT Healthcare Projects
- Biomedical Engineering Education
- Wearable Device Simulations
- Sensor Interfacing Practice



Ans48.

Introduction

The Government of Jammu & Kashmir is working towards developing Jammu as a Smart City by integrating Internet of Things (IoT) technologies into public infrastructure. IoT enables real-time monitoring, automation, and efficient resource management. This proposal presents three IoT-based solutions for Jammu Smart City: Smart Street Lighting, Smart Parking Management, and Air Quality Monitoring. These systems aim to reduce energy consumption, improve traffic management, and enhance environmental monitoring.

1. Smart Street Lighting

The LDR measures ambient light levels. During daytime, street lights remain OFF. At night, the PIR sensor detects pedestrians or vehicles and turns the street light ON only when movement is detected, reducing electricity consumption.

Estimated Cost

Component	Cost (INR)
ESP32	₹500
LDR	₹40
PIR Sensor	₹120
Relay	₹100
LED Lamp	₹250

Total Cost: ₹1,010 per node

2. Smart Parking Management

Each parking slot is equipped with an ultrasonic sensor to detect vehicle presence. The ESP32 uploads the occupancy status to the cloud. Drivers can check parking availability through a web dashboard before reaching the parking area.

Estimated Cost

Component	Cost (INR)
-----------	------------

ESP32	₹500
-------	------

HC-SR04	₹120
---------	------

Indicator LED	₹20
---------------	-----

Total Cost: ₹640 per parking node

3. Air Quality Monitoring

The MQ135 continuously monitors air pollution while the DHT11 records temperature and humidity. The collected data is transmitted to the cloud, where city officials can monitor pollution levels and identify highly polluted areas.

Estimated Cost

Component	Cost (INR)
ESP32	₹500
MQ135	₹220
DHT11	₹150

Total Cost: ₹870 per node

Benefits

- Reduces electricity consumption.
- Improves parking management.
- Provides real-time environmental monitoring.
- Supports data-driven decision making.

- Low installation and maintenance costs.
- Easily scalable for city-wide deployment.

Conclusion

The proposed IoT-based Smart City system integrates smart street lighting, parking management, and air quality monitoring using ESP32 controllers and Wi-Fi connectivity. These systems provide real-time monitoring, reduce operational costs, improve public services, and contribute to the development of a sustainable and efficient Smart City in Jammu.

Arduino Code (Google Drive)

<https://drive.google.com/drive/folders/1PCpKltWAr4kl7JgfFlfpIGKMGQMZsc6q?usp=sharing>

Ans49.

Introduction

IoT devices are connected to the internet and often exchange sensitive information. If proper security measures are not followed, attackers can steal data, gain unauthorized access, or even take control of the device. The given ESP8266 program contains several common security issues that should be fixed before deploying the device in a real environment.

Security Vulnerabilities

Vulnerability	Risk	Recommended Solution
Hardcoded Wi-Fi SSID and Password	Anyone who accesses the source code can view the Wi-Fi credentials and connect to the network.	Store credentials in a separate config.h file and exclude it using .gitignore.
Using HTTP Instead of HTTPS	Data is transmitted in plain text and can be intercepted by attackers.	Use HTTPS with SSL/TLS encryption.
Public MQTT Broker Without Authentication	Anyone can publish or subscribe to the MQTT topics, leading to fake data or unauthorized access.	Use MQTT authentication and TLS encryption.
No Validation of Incoming Commands	Attackers can send malicious commands and control the IoT device.	Validate all received commands before executing them.
No Data Encryption	Sensor data can be read or modified during transmission.	Encrypt communication using SSL/TLS.
Connecting to Public Wi-Fi	Public networks are less secure and vulnerable to attacks.	Use a trusted Wi-Fi network with WPA2/WPA3 security.

Corrected Code 1 – Secure Wi-Fi Credentials

config.h

```
#ifndef CONFIG_H
#define CONFIG_H

const char* ssid = "YOUR_WIFI_NAME";
const char* password = "YOUR_WIFI_PASSWORD";

#endif
```

Main Program

```
#include <ESP8266WiFi.h>
#include "config.h"

void setup() {

    Serial.begin(115200);

    WiFi.begin(ssid, password);

    while (WiFi.status() != WL_CONNECTED) {

        delay(500);

        Serial.print(".");
    }

    Serial.println("Wi-Fi Connected");

}
```

Explanation:

Instead of storing the Wi-Fi credentials directly in the main program, they are placed in a separate config.h file. This file can be excluded from GitHub using .gitignore, reducing the risk of exposing sensitive information.

Corrected Code 2 – Using HTTPS

Insecure HTTP

```
HTTPClient http;
```

```
http.begin("http://example.com/data");
```

Secure HTTPS

```
#include <WiFiClientSecure.h>
```

```
WiFiClientSecure client;
```

```
HTTPClient https;
```

```
client.setInsecure(); // For testing only
```

```
https.begin(client, "https://example.com/data");
```

```
int responseCode = https.GET();
```

```
https.end();
```

Explanation:

HTTPS encrypts the communication between the ESP8266 and the server. This prevents attackers from intercepting or modifying the transmitted data.

Additional Security Recommendations

- Use WPA3 or WPA2-Enterprise for Wi-Fi security.
- Enable MQTT authentication using a username and password.
- Use MQTT over TLS (port 8883) instead of an unsecured connection.
- Validate every command received from the network before executing it.
- Keep ESP8266 firmware and libraries updated to fix known security vulnerabilities.
- Protect API keys and passwords by storing them securely instead of hardcoding them.

Ans50.

Throughout this IoT course, I gained practical knowledge about how smart devices collect data, communicate with each other, and automate real-world tasks. Before starting these assignments, I knew the basic concepts of microcontrollers, but I had very little experience with integrating sensors, wireless communication, and cloud services. Working on these projects helped me understand how different hardware and software components come together to build useful IoT applications.

During the course, I worked on a variety of projects such as weather monitoring, smart home air quality monitoring, automated plant watering, smart door lock with OTP authentication, PIR-based security systems, flood detection, health monitoring, and smart city applications. Each project introduced me to different sensors like the DHT11, MQ-2 gas sensor, HC-SR04 ultrasonic sensor, PIR motion sensor, LDR, keypad, servo motor, and MAX30102 simulation. I also learned how to use communication technologies such as Wi-Fi, Bluetooth, HTTP, MQTT, and REST APIs to exchange data between devices and cloud platforms.

One of the most valuable lessons I learned was the importance of designing systems that are reliable and efficient. For example, using rolling averages helped reduce noisy sensor readings, while using the `millis()` function instead of `delay()` made programs more responsive. I also learned that IoT development is not only about connecting sensors but also about writing clean code, handling errors, and considering security. Topics such as HTTPS communication, MQTT authentication, and protecting Wi-Fi credentials showed me how important cybersecurity is in IoT applications.

As a Computer Science student, I believe these IoT skills will be very useful in my future career. Modern software development increasingly involves smart devices, cloud computing, and automation. Understanding how embedded systems interact with web applications and cloud services gives me an advantage in developing complete end-to-end solutions. Whether I work in software engineering, smart infrastructure, or industrial automation, the knowledge gained from this course will help me design intelligent systems that solve practical problems.

If I were to build my own IoT project in the future, I would develop a **Smart Hostel Management System** for university campuses. The system would include RFID or fingerprint authentication for attendance and room access, environmental sensors to monitor room temperature and air quality, smart energy meters to track electricity usage, and water level sensors for overhead tanks. ESP32 controllers would collect data from all

sensors and send it to a cloud server using Wi-Fi or LoRa for long-range communication. Students and hostel staff could access the information through a mobile application or web dashboard, receive maintenance alerts, monitor resource consumption, and improve overall hostel management. Such a system could reduce energy waste, improve security, and provide a better living environment for students.

Overall, this course has given me a strong foundation in IoT concepts, hardware interfacing, networking, and cloud integration. More importantly, it has shown me how technology can be used to solve real-life problems in areas such as agriculture, healthcare, smart homes, security, and smart cities. I look forward to building more advanced IoT solutions by combining embedded systems, artificial intelligence, and cloud computing in future projects.